LAPORAN PRAKTIKUM

STRUKTUR DATA

“Double Linked List”

disusun oleh: Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, dengan Judul "Double Linked List"

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 14 Mei 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 2

1.1 Latar Belakang 2

1.2 Tujuan 2

1.3 Manfaat 2

BAB II PEMBAHASAN 3

2.2 Implementasi Pada Java 3

2.2.1 NodeDLL_2511533020 3

2.2.2 InsertDLL_2511533020 5

2.2.3 PenelusuranDLL_2511533020 5

2.2.4 HapusDLL_2511533020 5

BAB III PENUTUP 6

3.1 Kesimpulan 6

3.2 Saran 6

3.3 Penutup 6

DAFTAR PUSTAKA 11

TUGAS PRAKTIKUM 12

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada praktikum minggu ini, materi yang dipelajari adalah Double Linked List (DLL). Berbeda dengan Single Linked List yang hanya memiliki satu pointer (next), Double Linked List memiliki dua pointer yaitu next dan prev. Pointer next digunakan untuk menunjuk ke node berikutnya, sedangkan pointer prev digunakan untuk menunjuk ke node sebelumnya. Dengan adanya dua pointer ini, penelusuran data dapat dilakukan dari dua arah yaitu dari depan ke belakang maupun dari belakang ke depan.

Kelebihan ini membuat DLL lebih fleksibel dibandingkan SLL, meskipun membutuhkan memori yang lebih besar karena setiap node menyimpan dua referensi. Pada praktikum ini, saya ditugaskan untuk mengimplementasikan berbagai operasi dasar pada DLL seperti penambahan node (di awal, akhir, dan posisi tertentu), penghapusan node, serta penelusuran maju dan mundur.Pada praktikum kali ini, SLL dipelajari melalui berbagai operasi dasar seperti penambahan node di depan, di akhir, dan di posisi tertentu, pencarian data, penghapusan node, serta penelusuran (traversal) seluruh elemen.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami konsep Double Linked List dan perbedaannya dengan Single Linked List. Saya juga diharapkan mampu mengimplementasikan operasi penambahan node di berbagai posisi (insertBegin, insertEnd, insertAtPosition), operasi penghapusan node (deleteHead, deleteLast, deletePos), serta operasi penelusuran data dari dua arah (forward dan backward traversal).

1.3 Manfaat

Manfaat yang saya peroleh dari praktikum ini antara lain memahami cara kerja pointer ganda pada struktur data dinamis. DLL banyak digunakan dalam aplikasi nyata seperti fitur back dan forward pada browser, navigasi playlist musik, atau sistem undo-redo. Dengan memahami DLL, saya juga lebih siap untuk mempelajari struktur data yang lebih kompleks seperti Circular Double Linked List.

\Selain itu, praktikum ini melatih saya dalam melakukan debugging karena kesalahan pengaturan pointer pada DLL sering menyebabkan program error.

BAB II
PEMBAHASAN

Double Linked List adalah struktur data linear yang terdiri dari kumpulan node. Setiap node pada DLL memiliki tiga komponen yaitu data (nilai yang disimpan), pointer next (menunjuk ke node setelahnya), dan pointer prev (menunjuk ke node sebelumnya). Node pertama disebut head, sedangkan node terakhir disebut tail. Pointer next pada tail bernilai null, dan pointer prev pada head juga bernilai null. Karena memiliki dua pointer, DLL memungkinkan penelusuran data dari dua arah, yaitu maju (dari head ke tail) dan mundur (dari tail ke head).

Kekurangan DLL dibandingkan SLL adalah penggunaan memori yang lebih besar dan waktu eksekusi yang sedikit lebih lama karena harus mengelola dua pointer setiap kali melakukan operasi insert atau delete.

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program Stack yang telah dipraktikkan.

2.2.1 NodeDLL_2511533020

```java
package pekan6_2511533020;

public class NodeDLL_2511533020 {
        //mendefinisikan kelas Node
    int data_3020; // data
    NodeDLL_2511533020 next_3020;
    NodeDLL_2511533020 prev_3020;

    public NodeDLL_2511533020(int data_3020) {
        this.data_3020 = data_3020;
        this.next_3020 = null;
        this.prev_3020 = null;
    }
}
```

(Gambar 2.2.1)

Kelas ini berfungsi sebagai cetakan untuk membuat node-node dalam Double Linked List. Terdapat tiga atribut yaitu data_3020 bertipe int untuk menyimpan nilai, next_3020 bertipe NodeDLL_2511533020 untuk menunjuk ke node berikutnya, dan prev_3020 bertipe NodeDLL_2511533020 untuk menunjuk ke node sebelumnya. Ketiga atribut tersebut diberi akhiran 3020 sesuai aturan penamaan. Konstruktor NodeDLL_2511533020(int data_3020) digunakan untuk menginisialisasi node baru dengan nilai tertentu, sekaligus mengatur next_3020 dan prev_3020 menjadi null.

2.2.2 InsertDLL_2511533020

```java
package pekan6_2511533020;

public class InsertDLL_2511533020 {
    //menambahkan node di awal DLL
    static NodeDLL_2511533020 insertBegin(NodeDLL_2511533020 head_3020, int data_3020) {
        //buat node baru
        NodeDLL_2511533020 new_node = new NodeDLL_2511533020(data_3020);
        // jadikan pointer nextnya head
        new_node.next_3020 = head_3020;
        //jadikan pointer prev head ke new_node
        if (head_3020 != null) {
            head_3020.prev_3020 = new_node;
        }
        return new_node;
    }
        //fungsi menambahkan node di akhir
    public static NodeDLL_2511533020 insertEnd(NodeDLL_2511533020 head_3020, int newData_3020) {
        //buat node baru
        NodeDLL_2511533020 newNode = new NodeDLL_2511533020(newData_3020);
        //jika dll null jadikan head
        if (head_3020 == null) {
            head_3020 = newNode;
        }
        else {
            NodeDLL_2511533020 curr = head_3020;
            while (curr.next_3020 != null ) {
                curr = curr.next_3020;
            }
            curr.next_3020 = newNode;
            newNode.prev_3020 = curr;
        }
        return head_3020;
    }
        //fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511533020 insertAtPosition(NodeDLL_2511533020 head_3020, int pos_3020, int new_data_3020) {
        // buat node baru
        NodeDLL_2511533020 new_node = new NodeDLL_2511533020(new_data_3020);
        if (pos_3020 == 1) {
```

```java
            new_node.next_3020 = head_3020;
            if (head_3020 != null) {
            head_3020.prev_3020 = new_node;
        }
            head_3020 = new_node;
            return head_3020;
    }

        NodeDLL_2511533020 curr = head_3020;
        for (int i_3020 = 1; i_3020 < pos_3020 - 1 && curr != null; ++i_3020) {
            curr = curr.next_3020;
            }
        if (curr == null) {
            System.out.println("Posisi Tidak ada");
            return head_3020;
            }
        new_node.prev_3020 = curr;
        new_node.next_3020 = curr.next_3020;
        curr.next_3020 = new_node;
        if (new_node.next_3020 != null) {
            new_node.next_3020.prev_3020 = new_node;
            }
        return head_3020;

        }
    public static void printList(NodeDLL_2511533020 head_3020) {
        NodeDLL_2511533020 curr = head_3020;
        while (curr != null) {
            System.out.print(curr.data_3020 + " <-> ");
            curr = curr.next_3020;

        }
        System.out.println();

    }
    public static void main(String[] args) {
        // membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511533020 head_3020 = new NodeDLL_2511533020(2) ;
```

```java
        head_3020.next_3020 = new NodeDLL_2511533020(3);
        head_3020.next_3020.prev_3020 = head_3020;
        head_3020.next_3020.next_3020 = new NodeDLL_2511533020(5);
        head_3020.next_3020.next_3020.prev_3020 = head_3020.next_3020;
        //cetak DLL awal
        System.out.print("DLL Awal: ");
        printList(head_3020);
        //tambah 1 di awal
        head_3020 = insertBegin(head_3020, 1);
        System.out.print(
                "simpul 1 ditambah di awal: ");
        printList(head_3020);
        //tambah 6 di akhir
        System.out.print(
                "Simpul 6 ditambah di akhir: ");
        int data_3020 = 6;
        head_3020 = insertEnd(head_3020, data_3020);
        printList(head_3020);
        // menambah node 4 di posisi 4
        System.out.print("Tambah node 4 di Posisi 4: ");
        int data2_3020 = 4;
        int pos_3020 = 4;
        head_3020 = insertAtPosition(head_3020, pos_3020 ,data2_3020);
        printList(head_3020);


    }
}
```

(Gambar 2.2.)

Kelas ini berisi tiga method untuk menambahkan node ke dalam Double Linked List. Method insertBegin digunakan untuk menambahkan node di posisi paling depan. Langkahnya

adalah membuat node baru, mengarahkan pointer next node baru ke head yang lama, kemudian jika head tidak kosong, pointer prev dari head lama diarahkan ke node baru. Selanjutnya node baru dikembalikan sebagai head yang baru. Method insertEnd digunakan untuk menambahkan node di posisi paling belakang. Jika list kosong, node baru langsung menjadi head. Jika tidak, program mencari node terakhir dengan perulangan while, lalu pointer next dari node terakhir diarahkan ke node baru, dan pointer prev dari node baru diarahkan ke node terakhir tersebut.

Method insertAtPosition digunakan untuk menambahkan node pada posisi tertentu. Jika posisi 1, maka method akan memanggil insertBegin. Jika posisi lebih dari 1, program menelusuri list sampai node sebelum posisi yang dituju, lalu pointer-pointernya diatur sedemikian rupa sehingga node baru tersisip di antara dua node yang sudah ada. Pada method main, dibuat DLL awal berisi 2,3,5. Selanjutnya dilakukan penambahan node 1 di awal, node 6 di akhir, dan node 4 di posisi 4. Hasil akhirnya DLL berisi 1,2,3,4,5,6.

```
Console
<terminated> InsertDLL_2511533020 [Java Application] C:\Users\denni\.p2\poo
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di Posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6
```

2.2.3 PenelusuranDLL_2511533020

```java
package pekan6_2511533020;

public class PenelusuranDLL_2511533020 {
    //fungsi penelusuran maju
    static void forwardTraversal(NodeDLL_2511533020 head_3020) {
        //memulai penelusuran head
        NodeDLL_2511533020 curr = head_3020;
        //lanjutkan sampai akhir
        while (curr != null) {
            //print data
            System.out.print(curr.data_3020 + " <-> ");
            //pindah ke node berikutnya
            curr = curr.next_3020;
        }
        //print spasi
        System.out.println();
    }
    //fungsi penelusuran mundur
    static void backwardTraversal (NodeDLL_2511533020 tail_3020) {
        // mulai dari akhir
        NodeDLL_2511533020 curr = tail_3020;
        //lanjut sampai head
        while (curr != null) {
            //cetak data
            System.out.print(curr.data_3020 + " <-> ");
            //pindah ke node sebelumnya
            curr = curr.prev_3020;
        }
        //cetak spasi
        System.out.println();

    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        // cetak DLL
        NodeDLL_2511533020 head_3020 = new NodeDLL_2511533020(1);
        NodeDLL_2511533020 second_3020 = new NodeDLL_2511533020(2);
```

```java
        NodeDLL_2511533020 third_3020 = new NodeDLL_2511533020(3);

        head_3020.next_3020 = second_3020;
        second_3020.prev_3020 = head_3020;
        second_3020.next_3020 = third_3020;
        third_3020.prev_3020 = second_3020;

        System.out.println("Penelusuran Maju:");
        forwardTraversal(head_3020);

        System.out.println("Penelusuran Mundur:");
        backwardTraversal(third_3020);

    }

}
```

(Gambar 2.2.3)

Kelas ini berisi dua method untuk melakukan penelusuran data pada Double Linked List. Method forwardTraversal digunakan untuk menampilkan data dari depan ke belakang. Penelusuran dimulai dari head, kemudian selama node tidak null, data dicetak dan pointer berpindah ke node berikutnya. Method backwardTraversal digunakan untuk menampilkan data dari belakang ke depan.

Penelusuran dimulai dari tail (node terakhir), kemudian selama node tidak null, data dicetak dan pointer berpindah ke node sebelumnya. Pada method main, dibuat DLL sederhana berisi 1,2,3. Penelusuran maju menghasilkan 1,2,3, sedangkan penelusuran mundur menghasilkan 3,2,1.

```
Console
<terminated> PenelusuranDLL_2511533020 [Java Application] C:\Users\
Penelusuran Maju:
1 <-> 2 <-> 3 <->
Penelusuran Mundur:
3 <-> 2 <-> 1 <->
```

2.2.4 HapusDLL_2511533020

```java
package pekan6_2511533020;

public class HapusDLL_2511533020 {
    // fungsi menghapus node awal
    public static NodeDLL_2511533020 de1Head(NodeDLL_2511533020 head_3020) {
        if (head_3020 == null) {
            return null;        }
        NodeDLL_2511533020 temp = head_3020;
        head_3020 = head_3020.next_3020;
        if (head_3020 != null)      {
            head_3020.prev_3020 = null;     }
        return head_3020;
        }
    //fungsi menghapus di akhir
    public static NodeDLL_2511533020 de1Last(NodeDLL_2511533020 head_3020) {
        if (head_3020 == null) {
            return null;        }
        if (head_3020.next_3020 == null)    {
            return null;        }
        NodeDLL_2511533020 curr = head_3020;
        while (curr.next_3020 != null) {
            curr = curr.next_3020;
        }
        // update pointer previous node
        if (curr.prev_3020 != null) {
            curr.prev_3020.next_3020 = null;    }
        return head_3020;
        }
    // fungsi menghapus node posisi tertentu
    public static NodeDLL_2511533020 de1Pos(NodeDLL_2511533020 head_3020, int pos_3020) {
        // jika DLL kosong
        if (head_3020 == null) {
            return head_3020;   }
        NodeDLL_2511533020 curr = head_3020;
        // telusuri sampai ke node yang akan dihapus
        for (int i_3020 = 1; curr != null && i_3020 < pos_3020; ++i_3020) {
            curr = curr.next_3020;  }
        // jika posisi tidak ditemukan
```

```java
        if (curr == null) {
            return head_3020;   }
        // Update pointer
        if (curr.next_3020 != null) {
            curr.prev_3020.next_3020 = curr.next_3020;  }
        if (curr.next_3020 != null) {
            curr.next_3020.prev_3020 = curr.prev_3020;  }
        // Jika yang dihapus head
        if (head_3020 == curr) {
            head_3020 = curr.next_3020;     }
        return head_3020;

    }

    public static void printlist(NodeDLL_2511533020 head_3020) {
        NodeDLL_2511533020 curr = head_3020;
        while (curr != null) {
            System.out.print(curr.data_3020 + " ->");
            curr = curr.next_3020;
        }
        System.out.println();

    }
    public static void main(String[] args) {
        // buat sebuah DLL
        NodeDLL_2511533020 head_3020 = new NodeDLL_2511533020(1);
        head_3020.next_3020 = new NodeDLL_2511533020(2);
        head_3020.next_3020.prev_3020 = head_3020;
        head_3020.next_3020.next_3020 = new NodeDLL_2511533020(3);
        head_3020.next_3020.next_3020.prev_3020 = head_3020.next_3020;
        head_3020.next_3020.next_3020.next_3020 = new NodeDLL_2511533020(4);
        head_3020.next_3020.next_3020.next_3020.prev_3020 = head_3020.next_3020.next_3020;
        head_3020.next_3020.next_3020.next_3020.next_3020 = new NodeDLL_2511533020 (5);
        head_3020.next_3020.next_3020.next_3020.next_3020.prev_3020 =      head_3020.next_3020.next_3020.next_3020;

        System.out.print("DLL Awal: ");
        printlist(head_3020);
```

```java

        System.out.print("Setelah head dihapus: ");
        head_3020 = de1Head(head_3020);
        printlist(head_3020);

        System.out.print("Setelah node terakhir dihapus: ");
        head_3020 = de1Last(head_3020);
        printlist(head_3020);

        System.out.print("Menghapus Node Ke 2: ");
        head_3020 = de1Pos(head_3020, 2);

        printlist(head_3020);


    }

}
```

(Gambar 2.2.4)

Kelas ini berisi tiga method untuk menghapus node dari Double Linked List. Method de1Head digunakan untuk menghapus node paling depan. Caranya adalah dengan menggeser head ke node berikutnya, kemudian jika head baru tidak kosong, pointer prev-nya diatur menjadi null. Method de1Last digunakan untuk menghapus node paling belakang. Program mencari node terlebih dahulu, kemudian pointer next dari node sebelumnya diatur menjadi null.

Method de1Pos digunakan untuk menghapus node pada posisi tertentu. Program menelusuri list sampai node yang akan dihapus ditemukan, lalu mengupdate pointer next dari node sebelumnya agar menunjuk ke node setelahnya, dan pointer prev dari node setelahnya agar menunjuk ke node sebelumnya. Jika node yang dihapus adalah head, maka head digeser ke node berikutnya. Pada method main, dibuat DLL awal berisi 1,2,3,4,5. Selanjutnya dilakukan penghapusan head, penghapusan node terakhir, dan penghapusan node pada posisi 2. Hasil akhirnya DLL berisi 3,4.

```
Console
<terminated> HapusDLL_2511533020 [Java Application] C:\Users\den
DLL Awal: 1 ->2 ->3 ->4 ->5 ->
Setelah head dihapus: 2 ->3 ->4 ->5 ->
Setelah node terakhir dihapus: 2 ->3 ->4 ->
Menghapus Node Ke 2: 2 ->4 ->
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List memiliki kelebihan dibandingkan Single Linked List yaitu kemampuan untuk melakukan penelusuran dari dua arah. Hal ini dimungkinkan karena setiap node pada DLL memiliki dua pointer yaitu next dan prev.

Operasi penambahan dan penghapusan node pada DLL memerlukan pengaturan kedua pointer dengan benar agar koneksi antar node tetap terjaga. Dari keempat program yang telah dibuat, operasi insert, delete, dan traversal pada DLL telah berhasil diimplementasikan sesuai dengan ketentuan yang diberikan.

3.2 Saran

Untuk pengembangan lebih lanjut, sebaiknya program dilengkapi dengan validasi input yang lebih baik, misalnya ketika posisi yang dimasukkan melebihi jumlah node, program dapat memberikan opsi untuk menambah node di akhir.

Selain itu, pada method de1Last di kelas HapusDLL_2511533020, sebaiknya ditambahkan penanganan khusus untuk list yang hanya berisi satu node. Secara umum, semua program sudah mengikuti aturan penamaan dengan akhiran NIM 3020.

3.3 Penutup

Demikian laporan praktikum Double Linked List ini saya buat. Laporan ini berisi penjelasan mengenai konsep DLL, implementasi kode, serta analisis dari keempat program yang telah dibuat. Saya menyadari bahwa laporan ini masih memiliki kekurangan.

Oleh karena itu, kritik dan saran yang membangun sangat saya harapkan untuk perbaikan di masa mendatang.

DAFTAR PUSTAKA

[1] Oracle, "LinkedList (Java Platform SE 17)," Oracle. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/LinkedList.html (diakses: 14 Mei 2026).

[2] W3Schools, "Java LinkedList," W3Schools. [Online].Available: https://www.w3schools.com/java/java_linkedlist.asp (diakses: 14 Mei 2026).

[3] Wikipedia, "Doubly linked list," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Doubly_linked_list (diakses: 14 Mei 2026).

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022..

TUGAS PRAKTIKUM

- **KELAS ADT:**

```java
package pekan6;

public class Lagu_2511533020 {

       private String judul_3020;
    private String penyanyi_3020;
    private Lagu_2511533020 next_3020;
    private Lagu_2511533020 prev_3020;

    // Constructor
    public Lagu_2511533020(String judul_3020, String penyanyi_3020) {
        this.judul_3020 = judul_3020;
        this.penyanyi_3020 = penyanyi_3020;
        this.next_3020 = null;
        this.prev_3020 = null;
    }

    // Getter
    public String getJudul_3020() {
        return judul_3020;
    }

    public String getPenyanyi_3020() {
        return penyanyi_3020;
    }

    public Lagu_2511533020 getNext_3020() {
        return next_3020;
    }

    public Lagu_2511533020 getPrev_3020() {
        return prev_3020;
    }

    // Setter
    public void setJudul_3020(String judul_3020) {
        this.judul_3020 = judul_3020;
    }

    public void setPenyanyi_3020(String penyanyi_3020) {
        this.penyanyi_3020 = penyanyi_3020;
    }

    public void setNext_3020(Lagu_2511533020 next_3020) {
        this.next_3020 = next_3020;
    }

    public void setPrev_3020(Lagu_2511533020 prev_3020) {
        this.prev_3020 = prev_3020;
    }
```

```java
    public String toString() {
        return judul_3020 + " - " + penyanyi_3020;
    }
}
```

Kelas ini merepresentasikan satu buah lagu. Atributnya ada empat: judul_3020, penyanyi_3020, next_3020 (pointer ke lagu berikutnya), dan prev_3020 (pointer ke lagu sebelumnya). Semua atribut menggunakan akhiran 3020 sesuai aturan.

Kelas ini juga dilengkapi dengan constructor, getter, setter, dan method toString() untuk mencetak lagu dengan format "judul - penyanyi".

KELAS DRIVER

```java
package pekan6;

import java.util.Scanner;

public class Musik_2511533020 {
    private Lagu_2511533020 head_3020;
    private Lagu_2511533020 tail_3020;

    // Constructor
    public Musik_2511533020() {
        this.head_3020 = null;
        this.tail_3020 = null;
    }

    public void tambahLagu_3020(String judul, String penyanyi) {
        Lagu_2511533020 laguBaru = new Lagu_2511533020(judul, penyanyi);

        if (head_3020 == null) {

            head_3020 = laguBaru;
            tail_3020 = laguBaru;
        } else {
            // Tambah di akhir
            tail_3020.setNext_3020(laguBaru);
            laguBaru.setPrev_3020(tail_3020);
            tail_3020 = laguBaru;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }

       public void hapusLaguAwal_3020() {
        if (head_3020 == null) {
            System.out.println("Playlist kosong! Tidak ada lagu yang dapat dihapus.");
            return;
        }

        if (head_3020 == tail_3020) {

```

```java
            head_3020 = null;
            tail_3020 = null;
        } else {
            head_3020 = head_3020.getNext_3020();
            head_3020.setPrev_3020(null);
        }
        System.out.println("Lagu pertama berhasil dihapus!");
    }

    public void tampilMaju_3020() {
        if (head_3020 == null) {
            System.out.println("Playlist kosong! Belum ada lagu.");
            return;
        }

        System.out.println("\n=== PLAYLIST (Dari Awal ke Akhir) ===");
        Lagu_2511533020 current = head_3020;
        int urutan = 1;
        while (current != null) {
            System.out.println(urutan + ". " + current.toString());
            current = current.getNext_3020();
            urutan++;
        }
    }


    public void tampilMundur_3020() {
        if (tail_3020 == null) {
            System.out.println("Playlist kosong! Belum ada lagu.");
            return;
        }

        System.out.println("\n=== PLAYLIST (Dari Akhir ke Awal) ===");
        Lagu_2511533020 current = tail_3020;
        int urutan = 1;
        while (current != null) {
            System.out.println(urutan + ". " + current.toString());
            current = current.getPrev_3020();
```

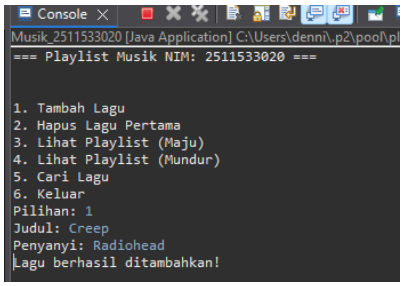
```java
            urutan++;
        }
    }


    public void cariLagu_3020(String judulCari) {
        if (head_3020 == null) {
            System.out.println("Playlist kosong! Tidak ada lagu untuk dicari.");
            return;
        }

        boolean ditemukan = false;
        Lagu_2511533020 current = head_3020;
        int posisi = 1;

        while (current != null) {

            if (current.getJudul_3020().equalsIgnoreCase(judulCari)) {
                System.out.println("Lagu ditemukan pada urutan ke-" + posisi);
                System.out.println("Judul: " + current.getJudul_3020());
                System.out.println("Penyanyi: " + current.getPenyanyi_3020());
                ditemukan = true;
                break;
            }
            current = current.getNext_3020();
            posisi++;
        }

        if (!ditemukan) {
            System.out.println("Lagu dengan judul \"" + judulCari + "\" tidak ditemukan.");
        }
    }


    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Musik_2511533020 playlist = new Musik_2511533020();
        int pilihan;
```

```java

        System.out.println("=== Playlist Musik NIM: 2511533020 ===\n");

        do {
            System.out.println("\n1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan = input.nextInt();
            input.nextLine();
            switch (pilihan) {
                case 1:
                    System.out.print("Judul: ");
                    String judul = input.nextLine();
                    System.out.print("Penyanyi: ");
                    String penyanyi = input.nextLine();
                    playlist.tambahLagu_3020(judul, penyanyi);
                    break;
                case 2:
                    playlist.hapusLaguAwal_3020();
                    break;
                case 3:
                    playlist.tampilMaju_3020();
                    break;
                case 4:
                    playlist.tampilMundur_3020();
                    break;
                case 5:
                    System.out.print("Masukkan judul lagu yang dicari: ");
                    String judulCari = input.nextLine();
                    playlist.cariLagu_3020(judulCari);
                    break;
                case 6:
                    System.out.println("Terima kasih telah menggunakan program ini.");
                    break;
```

```java
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 6);

        input.close();
    }
}
```

Kelas driver ini memiliki dua atribut utama: head_3020 (node pertama) dan tail_3020 (node terakhir). Method tambahLagu_3020() menambahkan lagu baru di akhir playlist dengan cara menghubungkan pointer next dari tail lama ke lagu baru, dan pointer prev dari lagu baru ke tail lama, lalu tail baru diupdate. Method hapusLaguAwal_3020() menghapus lagu pertama dengan cara menggeser head_3020 ke node berikutnya, lalu prev dari head baru diatur menjadi null. Jika playlist kosong, program akan menampilkan pesan.

Method tampilMaju_3020() menelusuri dari head ke tail dengan menggunakan pointer next_3020. Method tampilMundur_3020() menelusuri dari tail ke head dengan menggunakan pointer prev_3020. Inilah keunggulan Double Linked List. Method cariLagu_3020() mencari

lagu berdasarkan judul dengan perbandingan equalsIgnoreCase() sehingga tidak membedakan huruf besar dan kecil. Menu interaktif menggunakan perulangan do-while dengan 6 pilihan.

OUTPUT